CityTech TTP Summer Bootcamp

Intro to Data Analytics

2026-07-06

Agenda

1. Celebrity Name Chain
2. Review
3. Data Analytics
4. Jupyter Notebooks
5. pandas and EDA

Continue Building: Celebrity Name Chain

Pick up your **Celebrity Name Chain** project where you left off.

- Finish the **core** game first: the API rules and the polling client
- Then take on the **tiered learning goals**
- Full spec: `project_specs/celebrity-name-chain.md`

Review

What are the most important things you remember from last week?

The Purpose of Data Analytics

Data analytics turns **information about the past** into **better decisions about the future**.

- Study what has **already happened** to find **patterns** and **trends**
- Use those patterns to **predict** what is likely to happen next
- Let those predictions **guide the decisions** you make now

That prediction can come from a **human** or a **machine**:

- **Human expertise**: an analyst or **consultant** reads the data and advises
- **Machine learning**: models learn the patterns and predict at scale

Two Languages for Data Analytics

Most data analytics work happens in one of two languages: **R** and **Python**.

- They overlap a lot: both can load, analyze, and visualize data
- But they come from **different worlds**, each with its own strengths
- Let's look at each, then pick one for this program

R

R was built by statisticians, for statistics.

- **Origin**: created in academia for **statistical computing**
- **Strengths**: statistics and visualization (`tidyverse` , `ggplot2`)
- **Home turf**: academia and research (biostatistics, econometrics, social science)
- Feels different from general programming: `<-` assignment, 1-based indexing, formula syntax

R and the tidyverse

Read a CSV, filter rows, then summarize:

```
library(tidyverse)

sales <- read_csv("sales.csv")

sales |>
  filter(region == "West") |>
  group_by(product) |>
  summarize(total = sum(amount))
```

Python

Python is a general-purpose language that grew into data work.

- **Origin**: a **general-purpose** language (web, scripting, automation)
- **Strengths**: versatile - analysis, ML, automation, and glue code (`pandas` , `NumPy` , `scikit-learn`)
- **Home turf**: industry and software engineering
- Familiar to engineers: normal packages, testing, and objects
- **End to end**: one language from data cleaning to model to shipped app

Python and pandas

The same task in Python, with **pandas**:

```
import pandas as pd

sales = pd.read_csv("sales.csv")

(sales[sales["region"] == "West"]
    .groupby("product")["amount"]
    .sum())
```

Choosing Python

For this program, we're using **Python**. Why:

- You're becoming **software engineers**, and Python is the easiest on-ramp from coding
- **One language for everything**: analysis, ML, automation, even web
- The **most popular** language for data work, with a huge ecosystem and community
- Skills carry straight over to the rest of your engineering toolkit

Aside: Python vs R in the Job Market

The job data backs up choosing Python:

- Python appears in **far more** data postings than R - about **3:1 to 15:1** (varies by source)
- Python is consistently the **#1** requested data skill; R sits mid-pack
- Roughly **35-40%** of data-science postings mention **both** - often "and," not "or"
- R stays strong in its niches: **biostatistics, pharma, academia, finance**
- Pay is comparable (~$120-125k); Python's edge is the **volume of roles**

Numbers vary by source, but all point the same way: **Python well ahead**.

Sharing Data: CSV

Both R and Python read and write **CSV** natively (you just saw `read_csv` in each).

- Plain text and language-agnostic: export from one, load in the other
- A simple **intermediary** for handing data between tools, languages, and teammates
- Trade-off: CSV loses **types** (everything is text, so dates and numbers get re-inferred)
- A better solution: **Feather / Arrow**, created by the **pandas** and **tidyverse** teams together, which keeps **types** and is fast for sharing between Python and R

Scripts Run Top to Bottom

A Python script (`.py`) runs **top to bottom**, start to finish, every time you run it. This is **sequential execution** (running it all at once is sometimes called *batch* mode).

But **exploratory data analysis (EDA)** is iterative, like the **scientific method**:

hypothesize → test → revise → repeat

Re-running a long script from the top on every small change - reloading data, recomputing everything - is slow and **counterproductive** for that tight loop.

Aside: What Is EDA?

Exploratory Data Analysis (EDA) is your first, open-ended look at a dataset - getting to know it before drawing conclusions or building models.

- **Summarize**: shape, ranges, averages, missing values
- **Visualize**: plots that reveal patterns, trends, and outliers
- **Question**: form hunches and check them against the data
- The goal is to **understand** the data, and spot what's interesting or wrong

Jupyter Notebooks

A **notebook** is a document of **cells** you run one at a time, in any order - not top to bottom.

- Each cell runs on its own, and the notebook **keeps state in memory** between runs - as if **every variable were global**, shared across all cells
- So you re-run just the **cell you're working on**, without redoing everything above
- Mix **code**, its **output** (tables, charts), and **notes** (Markdown) in one document
- The `.ipynb` file **saves the outputs too**, not just the code (a `.py` holds only code)
- A perfect fit for the EDA loop: hypothesize, test, revise, repeat

Files end in `.ipynb`; run them in **Jupyter**, **VS Code**, or **Google Colab**.

Notebook Demo and Gotchas

Start Jupyter, then open `demos/notebook_basics.ipynb`:

```
jupyter notebook   # classic single-document view
jupyter lab        # fuller IDE: tabs, file browser, terminals
```

- **Out of order**: cells share memory, so an odd run order gives odd results
- **Stale outputs**: edit a cell but skip re-running the ones below, and their output no longer matches
- The fix: **Restart Kernel and Run All Cells** - runs everything top to bottom, like a script

Showing a Value in a Cell

A notebook **automatically displays the last expression** in a cell - no `print()` needed.

```
import random

numbers = []
numbers          # this bare line displays the list
```

- Jupyter shows the value of the **final line** when it's an expression
- That's why cells often end with just a name (`numbers` , `df`)
- **Rich display**: a DataFrame renders as a formatted **table**, plots show inline
- Only the **last** expression shows - use `print()` for earlier lines or several values

pandas

pandas is the standard Python library for working with **tabular data** (rows and columns).

- Load data from **CSV, Excel, SQL, JSON**, and more
- **Clean, filter, group, join, and summarize** it
- The backbone of **data analysis** and **EDA** in Python
- Built on **NumPy**; pairs with plotting and machine-learning libraries

By convention: `import pandas as pd`

Aside: NumPy

NumPy is the numerical foundation that pandas is built on.

- Provides the `ndarray` : a fast, fixed-type **n-dimensional array**
- Operates on whole arrays at once (**vectorized**), in fast C code, no Python loops
- pandas stores each column as a NumPy array under the hood
- You'll usually use pandas directly, but NumPy powers the speed

By convention: `import numpy as np`

The DataFrame

pandas' core type is the **DataFrame**: a 2-D **table**, like a spreadsheet or SQL table.

Labeled means each column has a **name** and each row has an **index** - so you grab data by label (`df["age"]`), not just by position.

	Python	JavaScript	pandas
A column	`list`	`Array`	`Series`
A row	`dict`	object `{}`	a row
The whole table	`list` of `dict`s	array of objects	**`DataFrame`**

A DataFrame stores data **by column** and analyzes it **without writing loops**.

Why Not Just Lists and Objects?

A `list` of `dict`s (or array of objects) works, but for real analysis it struggles:

- **Loops for everything**: filtering or summing means writing manual loops
- **Slow** on large data - processed row-by-row, no vectorization
- **No built-in analysis**: no `mean`, `groupby`, or joins out of the box

The **DataFrame** fixes it:

- **Vectorized**: `df[df.age > 30]` or `df["amount"].sum()`, no loops
- **Fast**: columnar storage on NumPy arrays, running at **C speed**
- **Batteries included**: `groupby()`, `describe()`, joins, missing-data handling

A Few EDA Tools in pandas

Just a taste of what pandas gives you for free:

- `df.head()` - peek at the first rows
- `df.info()` - columns, types, and non-null counts
- `df.describe()` - summary stats for the numeric columns
- `df["col"].value_counts()` - frequency of each value in a column
- `df.isna().sum()` - count missing values per column

The demo notebook explores these and more.

Demo: Squirrel Census EDA

Let's run a notebook together: `demos/eda-squirrel/squirrel_eda.ipynb`

- Real data: the **2018 Central Park Squirrel Census** (3,023 sightings)
- Walk the ritual: `head` → `info` → `describe` → missing values → `value_counts`
- Launch with `jupyter lab` (or `jupyter notebook`)

Decoding the Squirrel ID

Every squirrel has a `unique_squirrel_id` like `21B–AM–1019–04` - not random, but a **composite key**:

- `21B` - **hectare** (the park's grid cell)
- `AM` - **shift** (morning or afternoon)
- `1019` - **date** (MMDD: October 19)
- `04` - the **Nth squirrel** counted in that hectare that shift

How did we know? We read the dataset's **data dictionary**:

https://data.cityofnewyork.us/Environment/2018-Central-Park-Squirrel-Census-Squirrel-Data/vfnx-vebw

What Is a Data Dictionary?

A **data dictionary** describes a dataset: what each **column** means, its **type**, allowed values, and how codes or IDs are built.

- It tells you what the data *actually is*, so you don't have to guess
- Usually published with the data (a page, a PDF, or an attached doc)
- Reading it first saves you from misreading columns

Notes:

- It's the data world's version of **API docs** (like **OpenAPI / Swagger**)
- **Not always available**: sometimes you must infer meaning from the data itself

Today's Exit Ticket

Students should be able to:

- [ ] Explain the **purpose of data analytics**
- [ ] Define and **compare/contrast Python and R** for data analytics
- [ ] Explain what **EDA** is and identify some common analyses
- [ ] Explain what a **data dictionary** is
- [ ] Explain the purpose of, **open, and run** a Jupyter notebook
- [ ] Explain the purpose of **pandas**, and compare it to Python and JavaScript native data types